

Building a Home SOC Lab on Ubuntu Server

Deploying Wazuh SIEM and Suricata IDS in a VirtualBox lab, end to end, with real troubleshooting along the way.

LAB STACK

PLATFORM	Ubuntu (VirtualBox VM) + Docker
SIEM	Wazuh 4.9.0 — indexer, manager, dashboard
NETWORK IDS	Suricata 8.0.6, monitoring enp0s3
INTEGRATION	Wazuh Agent forwarding Suricata eve.json alerts
RESULT	Live, searchable IDS alerts in the Wazuh dashboard

Contents

- 1. Project Overview**
Goal, scope, and final stack summary
- 2. Phase 1 — Docker Engine Installation**
GPG key issues, repo codename mismatch, install
- 3. Phase 2 — Wazuh SIEM Deployment**
Cloning wazuh-docker, cert generation, DNS troubleshooting
- 4. Phase 3 — Disk & Memory Crisis Recovery**
Disk full at 100%, VM freeze, recovery-mode partition resize
- 5. Phase 4 — Suricata IDS Installation**
PPA setup, interface configuration, service verification
- 6. Phase 5 — Wazuh Agent & Suricata Integration**
Agent install, ossec.conf log forwarding config
- 7. Phase 6 — Verification & Live Alert Capture**
Nmap test traffic, confirmed alerts in Wazuh Discover
- 8. Skills Demonstrated**
Technical competencies shown throughout the build
- 9. Issues Log & Resolutions**
Full troubleshooting reference table
- 10. Next Steps**
Recommended follow-on lab work

1. Project Overview

This document records the build-out of a self-hosted Security Operations Center (SOC) lab environment, created for hands-on practice in a junior cybersecurity analyst role. The environment runs entirely inside a VirtualBox virtual machine and combines an open-source SIEM platform with a network intrusion detection system, giving a realistic detect → investigate → triage workflow similar to what a SOC analyst uses day to day.

Objective

Deploy a working SIEM (Wazuh) and network IDS (Suricata) on an Ubuntu VM, connect them so IDS alerts flow into the SIEM dashboard, and verify the pipeline with real generated network traffic.

Final Environment Summary

Component	Detail
Host / Hypervisor	Oracle VirtualBox
Guest OS	Ubuntu 26.04 (VM name: "javy")
Virtual Disk	Resized from 25 GB to 60 GB during the build
Allocated RAM	Increased to ~8.6 GB to support the full Wazuh stack
Containerization	Docker Engine + Docker Compose
SIEM	Wazuh 4.9.0 (indexer, manager, dashboard containers)
Network IDS	Suricata 8.0.6, monitoring interface enp0s3
Agent Integration	Wazuh Agent 4.9.0 forwarding /var/log/suricata/eve.json
Dashboard Access	https://localhost (self-signed certificate)

Result at a Glance

A live IDS alert (Suricata rule: **SURICATA STREAM ESTABLISHED invalid ack**, signature ID 2210029) generated by test scan traffic was captured by Suricata, parsed by the Wazuh agent's log collector, decoded and rule-matched by the Wazuh manager (rule ID 86601), indexed, and displayed as a searchable document in the Wazuh dashboard's Discover view — confirming the full pipeline works end to end.

PHASE 1

Docker Engine Installation

Setting up the container runtime that would host the Wazuh stack

The first step was installing Docker Engine and Docker Compose on the Ubuntu VM, since the Wazuh single-node deployment used later is distributed as a set of Docker containers.

Steps Performed

- Updated package lists and installed prerequisite packages (ca-certificates, curl, gnupg)
- Created the APT keyrings directory and downloaded Docker's official GPG signing key
- Added the Docker APT repository, pointing at the correct package architecture
- Ran `apt update` and installed `docker-ce`, `docker-ce-cli`, `containerd.io`, and `docker-compose-plugin`
- Added the working user to the `docker` group to allow running Docker commands without `sudo`

Issue Encountered — GPG Key Permission Errors

The initial one-line piped command (`curl ... | gpg --dearmor ...`) failed with `Permission denied`. The root cause: in a piped command, `sudo` placed before `curl` does not automatically apply to the second command in the pipe (`gpg`), which was therefore running as a regular user without write access to `/etc/apt/keyrings/`.

Fix: broke the pipeline into separate, individually-`sudo`'d steps for reliability:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /tmp/docker.asc
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg /tmp/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Issue Encountered — Unsupported Release Codename

The Ubuntu VM's release codename (`resolute`) was too new for Docker's official repository, which had no packages published for it yet. This caused `apt install` to fail with `"Package docker-ce has no installation candidate."`

Fix: manually rewrote the Docker sources list to target the nearest supported LTS codename (`noble`) instead of the VM's actual (too-new) codename:

```
echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu noble stable" | \
sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

Outcome

Docker Engine 29.7.2 and Docker Compose v5.5.0 installed and verified working without requiring `sudo` for day-to-day commands.

PHASE 2

Wazuh SIEM Deployment

Standing up the indexer, manager, and dashboard containers

Steps Performed

- Cloned the official wazuh-docker repository at the v4.9.0 tag
- Increased kernel `vm.max_map_count` to 262144 (required by the OpenSearch-based indexer)
- Generated SSL/TLS certificates for the indexer, dashboard, filebeat, and manager components
- Brought the full stack online with `docker compose up -d`

Issue Encountered — IPv6 Connectivity Failures

Certificate generation failed while pulling the certs-generator container image, with errors resolving down to `dial tcp [IPv6 address]:443: connect: connection refused`. The VM's VirtualBox NAT network was assigning IPv6 addresses that could not actually route out, and Docker was preferring them over IPv4.

Troubleshooting steps taken, in order:

- Set `{"ipv6": false}` in `/etc/docker/daemon.json` — did not fully resolve it, since this only affects container networking, not the host's own DNS resolution path
- Disabled IPv6 at the kernel level via `sysctl (net.ipv6.conf.all.disable_ipv6=1, etc.)` — surfaced a new, more specific error
- Diagnosed further and found the real fault: the host's default DNS server (an IPv6 address assigned via NAT/DHCP) was returning `SERVFAIL` for all lookups

Fix: pointed the network interface's DNS resolution directly at known-working public resolvers:

```
sudo resolvectl dns enp0s3 8.8.8.8 1.1.1.1
```

This diagnostic path is a good example of methodical troubleshooting: verifying DNS resolution independently of the failing tool (`nslookup . . . 8.8.8.8`) before assuming the problem was with Docker itself.

Outcome

All three Wazuh containers (indexer, manager, dashboard) started successfully and reported “Up” status. The dashboard was reachable at `https://localhost` and logged into successfully with the default admin credentials.

PHASE 3

Disk & Memory Crisis Recovery

The VM's 25 GB disk filled completely; recovered without data loss

After the Wazuh stack had been running for a period of time, the VM's graphical desktop (GDM) began repeatedly failing to start, and the system eventually became fully unresponsive, with the kernel logging a `soft lockup - CPU stuck for 80s watchdog error`.

Root Cause Diagnosis

Investigation with `df -h` and `docker system df` confirmed the VM's 25 GB virtual disk was completely full (0 bytes free). Docker's images, containers, and volumes for the Wazuh stack alone consumed roughly 17 GB, leaving no room for the desktop environment to write session files — which cascaded into the full system lockup.

Recovery Procedure

The recovery was performed without deleting any data, in this sequence:

- Powered off the VM cleanly via VirtualBox's Machine menu
- Resized the virtual disk container itself from 25 GB to 60 GB using `VBoxManage modifymedium disk ... --resize 61440`, run from the Windows host
- Booted the VM and, since the desktop still could not start (disk was still logically full), dropped into GRUB → Advanced options → Recovery Mode → root shell
- From the recovery root shell, verified the new unallocated space with `lsblk`
- Grew the actual partition to use the new space with `growpart`
- Expanded the filesystem to match with `resize2fs`
- Rebooted into a normal, fully working desktop session

```
growpart /dev/sda 2
resize2fs /dev/sda2
df -h # confirmed: 59G total, 32G free
```

Why This Mattered

This is a realistic operational scenario: a lab or production host running out of disk space due to log/data growth is a common real-world SIEM/IDS problem. The recovery demonstrates comfort working from a non-graphical recovery shell, safely resizing a live system's storage, and verifying each step before moving to the next — rather than reflexively reinstalling the VM from scratch.

Outcome

Disk capacity increased from 25 GB to 59 GB usable, with 32 GB free after recovery. All three Wazuh containers came back online automatically after reboot with zero data loss.

PHASE 4

Suricata IDS Installation

Adding live network traffic inspection to the lab

Steps Performed

- Added the official Suricata-stable PPA and installed Suricata 8.0.6 via APT
- Identified the VM's active network interface (enp0s3) using `ip a`
- Edited `/etc/suricata/suricata.yaml` to point the af-packet capture engine at enp0s3 (default config referenced the generic name eth0, which does not exist on modern predictable-naming Ubuntu systems)
- Updated Suricata's rule set with `suricata-update` (68,643 rules loaded)
- Enabled and started the `suricata systemd` service

Note on the PPA and Codename Mismatch

Given the earlier Docker repository issue with the `resolute` codename, the same problem was anticipated here. However, the Suricata-stable PPA turned out to have packages published for `resolute` directly, so `apt update` succeeded without needing the codename-substitution workaround used for Docker. This was confirmed by inspecting the generated sources file before proceeding, rather than assuming success.

Verification

```
sudo systemctl status suricata

● suricata.service - Suricata IDS/IPS/NSM/FW daemon
Active: active (running)
...suricata: This is Suricata version 8.0.6 RELEASE running in SYSTEM mode
```

A minor non-blocking warning was noted (`unix-manager: failed to create socket directory /var/run/suricata/: Permission denied`) — this only affects the optional live rule-reload management socket, not packet capture or alert generation, so it was documented and left for later cleanup rather than blocking progress.

Outcome

Suricata running as a persistent systemd service, actively capturing and analyzing traffic on the VM's primary network interface, with a full current ruleset loaded.

PHASE 5

Wazuh Agent & Suricata Integration

Forwarding IDS alerts from Suricata into the Wazuh SIEM

Steps Performed

- Downloaded and installed the Wazuh Agent package (v4.9.0) directly on the VM, configured to register with the local Wazuh manager at 127.0.0.1 (the manager container's host-mapped address)
- Edited `/var/ossec/etc/ossec.conf` to add a new `<localfile>` log-monitoring block pointing at Suricata's EVE JSON output
- Reloaded `systemd` and enabled/started the `wazuh-agent` service

Configuration Added

```
<localfile>
<log_format>json</log_format>
<location>/var/log/suricata/eve.json</location>
</localfile>
```

Issue Encountered — Filename Truncation on Download

A minor but instructive issue: the initial `curl -o wazuh-agent.deb` command saved the file as `wazuh-agentdeb` (the file extension's dot was dropped during input), which caused the subsequent `dpkg -i` step to fail with *"No such file or directory."* Confirmed the actual filename with `ls -la` before retrying, and installed successfully by referencing the file exactly as it existed on disk rather than assuming the intended name.

Verification

```
sudo systemctl status wazuh-agent

● wazuh-agent.service - Wazuh agent
Active: active (running)
Started wazuh-execd, wazuh-agentd, wazuh-syscheckd,
wazuh-logcollector, wazuh-modulesd
```

The presence and clean startup of `wazuh-logcollector` confirmed the agent was actively watching for and ready to forward the configured Suricata log file.

Outcome

Wazuh Agent installed, registered with the manager, and configured to tail and forward Suricata's `eve.json` event stream in real time.

PHASE 6

Verification & Live Alert Capture

Proving the full pipeline works with real generated traffic

Test Method

An `nmap -sV -A` scan (service/version detection, OS fingerprinting, and traceroute) was run against the VM's actual network-facing IP address (10.0.2.15), rather than `localhost`. This distinction mattered: traffic against `localhost` traverses the loopback interface, which Suricata was not configured to monitor — only the network-facing interface (`enp0s3`) was. Recognizing this difference and adjusting the test accordingly was key to getting a valid result.

Confirmation Steps

- Confirmed raw events were being written locally: `sudo wc -l /var/log/suricata/eve.json` → 1,509 lines logged
- Searched specifically for alert-type events (as opposed to routine flow/stats telemetry) using `grep '"event_type": "alert"'`
- Opened the Wazuh dashboard's **Discover** view and searched for `suricata` across all indexed data

Result

13 matching documents were found in the Wazuh dashboard, including a fully decoded IDS alert with the following key fields:

Field	Value
Wazuh rule description	Suricata: Alert - SURICATA STREAM ESTABLISHED invalid ack
Wazuh rule ID	86601
Wazuh rule level	3
Rule groups	ids, suricata
Suricata signature ID	2210029
Suricata alert severity	3
Source IP / Port	10.0.2.15 / 38382
Capture interface	enp0s3
Log source path	/var/log/suricata/eve.json

This confirms every link in the chain functioned correctly:

Suricata detection → **eve.json** → **Wazuh Agent log collector** → **Wazuh Manager rule decoder** → **Wazuh Indexer** → **Wazuh Dashboard (searchable, visualized)**.

8. Skills Demonstrated

Technical Skills

- **Linux system administration:** package management (APT), systemd service management, kernel parameter tuning, filesystem/partition management
- **Containerization:** Docker Engine and Docker Compose installation and multi-container orchestration
- **SIEM deployment:** Wazuh indexer/manager/dashboard configuration, certificate generation, agent registration
- **Network IDS:** Suricata installation, interface configuration, rule set management
- **Log pipeline integration:** configuring cross-tool log forwarding via `ossec.conf`
- **Virtualization management:** VirtualBox VM resource allocation, virtual disk resizing, snapshot-free recovery
- **Networking / DNS troubleshooting:** diagnosing IPv6 vs IPv4 resolution failures, working with `resolvectl` and `sysctl`

Operational / Analyst Skills

- **Root-cause troubleshooting:** did not stop at the first visible error; traced multiple issues (GPG permissions, DNS resolution, disk exhaustion) back to their actual underlying cause before applying a fix
- **Disaster recovery under pressure:** recovered a fully unresponsive VM without data loss by working through GRUB recovery mode and a non-graphical root shell
- **Verification discipline:** confirmed each step's success independently (e.g., checking DNS resolution against a known-good server before assuming the fix worked) rather than assuming success
- **Realistic test design:** recognized that a loopback-only scan would not test the network IDS correctly, and adjusted the test target accordingly
- **End-to-end pipeline validation:** did not stop at "the service is running" — validated actual alert data flowing all the way to the SIEM's search interface

9. Issues Log & Resolutions

A consolidated reference of every problem encountered during the build and how each was resolved.

#	Issue	Root Cause	Resolution
1	GPG key: "Permission denied" creating docker.gpg	sudo docker.gpg command only applied to \$, if first separate (sudo) then gpg	Self first separate (sudo) then gpg download key to /tmp, then sudo gpg --
2	Docker: "no installation candidate"	VM's release codename (resolute) too new	Reverted to published distro by distro noble (nearest supported LTS code
3	docker.list write failed: "No such file or directory"	Malformed multi-line command with escaped characters	Reduces to single line command and verified with cat before proceeding
4	Cert generation: IPv6 connection refused	VirtualBox NAT settings gave a non-routable IP address	Disabled IPv6 that Docker preferred config and at the kernel level
5	DNS lookups failing: SERVFAIL from 10.0.0.1	Host's default upstream DNS server (an IP address) not DNS capable	Pointed DNS to DNS server at 8.8.8.8 / 1.1.1.1 via resolvectl
6	VM disk 100% full; desktop (GDM) failed to start	VM installed Docker images & volumes (~17 GB) exceeded 40 GB disk	Resized VM to 60 GB, Ball 25 GB, grew disk partition and filesystem from reco
7	Suricata af-packet config referenced nonexistent interface	Default config used legacy interface naming	Edited af-packet config (resp03).yaml to match actual interface
8	wazuh-agent.deb install failed: file not found	Download filename dropped a character, saving the file as wazuh-agent	Fixed filename with -la and referenced it exactly for dpkg -i
9	First nmap test produced no IDS alerts	Scanner targeted localhost, which uses the loopback interface	Replaced the scanner with the VM's external interface IP (10.0.2.15) instead

10. Next Steps

With the core SIEM + IDS pipeline verified and working, the following extensions are planned to deepen the lab and produce more realistic analyst scenarios:

■ Add a deliberately vulnerable target

Deploy Metasploitable2 or DVWA on an isolated internal network so the lab has realistic attack traffic to detect, rather than only self-generated scan traffic.

■ Build Suricata-specific dashboards

Create dedicated Wazuh visualizations for Suricata alert volume, severity breakdown, and top source IPs, instead of relying solely on the generic Discover search.

■ Write and test custom Suricata rules

Author detection rules for specific attacker behaviors and confirm they fire correctly against controlled test traffic.

■ Resolve the Suricata unix-manager socket warning

Fix the `/var/run/suricata/` permissions issue to enable live rule-reload management.

■ Add host-based monitoring depth

Expand Wazuh's File Integrity Monitoring and Vulnerability Detection modules, which were confirmed present in the dashboard but not yet actively exercised.

■ Practice full incident write-ups

For each future detected alert, document the triage process: initial alert, investigation steps, root cause, and remediation — mirroring real SOC analyst reporting.

This report documents an ongoing personal training lab. All configurations shown were run in an isolated virtual machine for educational purposes.